

Activité : Partition parfaite

Table des matières

1	Description du problème	1
2	Partition parfaite des entiers de 1 à n	4
3	Algorithme glouton	4
4	Algorithme glouton répété	6
5	Programmation dynamique	8
6	Tester et Générer	11
7	Problèmes connexes	11

1 Description du problème

Définition 1.1 – Sac d’entiers

Un *sac d’entiers* est une collection d’entiers naturels dans laquelle un même nombre peut apparaître plusieurs fois.

Par exemple, le sac

$$S = \{1, 2, 2, 5\}$$

contient quatre entiers et le nombre 2 apparaît deux fois.

Plus généralement, un sac S de n entiers naturels peut être noté

$$S = \{s_1, s_2, \dots, s_n\}$$

où chaque s_i est un entier strictement positif.

Définition 1.2 – Partition d'un sac

Une *partition* d'un sac S consiste à séparer les éléments de S en deux sacs non vides S_1 et S_2 . Chaque élément du sac initial doit apparaître dans *exactement un* des deux sacs. Autrement dit :

- S_1 et S_2 sont non vides ;
- S_1 et S_2 sont disjoints (aucun entier n'apparaît dans les deux sacs) ;
- S_1 et S_2 recouvrent S : tous les éléments de S apparaissent dans S_1 ou S_2 .

Exemple – Partition d'un ensemble

Soit un sac $S = \{1, 2, 3, 4, 5\}$.

- Les sacs S_1 et S_2 forment une partition de S .
 - $S_1 = \{1\}$ et $S_2 = \{2, 3, 4, 5\}$
 - $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$
- Les sacs S_1 et S_2 ne forment pas une partition de S .
 - $S_1 = \{1, 2, 3, 4, 5\}$ et $S_2 = \emptyset$, car S_2 est vide.
 - $S_1 = \{1, 2, 3\}$ et $S_2 = \{3, 4, 5\}$, car le nombre 3 apparaît dans les deux sacs.
 - $S_1 = \{1, 2\}$ et $S_2 = \{4, 5\}$, car le nombre 3 n'apparaît dans aucun des deux sacs.

Définition 1.3 – Partition parfaite d'un sac pair

Un sac d'entiers S est dit *pair* si la somme de tous ses entiers est un nombre pair.

Une *partition parfaite* d'un sac pair est une partition (S_1, S_2) telle que la somme des entiers de S_1 est égale à la somme des entiers de S_2 .

Exemple – Partition parfaite d'un sac pair

$S = \{1, 2, 3, 4\}$ est un sac pair.

- $S_1 = \{1, 3\}$ et $S_2 = \{2, 4\}$ ne forment pas une partition parfaite, car $1 + 3 \neq 2 + 4$.
- $S_1 = \{1, 4\}$ et $S_2 = \{2, 3\}$ forment une partition parfaite $1 + 4 = 2 + 3$.

Définition 1.4 – Partition parfaite d'un sac impair

Un sac d'entiers S est dit *impair* si la somme de tous ses entiers est un nombre impair.

Une *partition parfaite* d'un sac impair est une partition (S_1, S_2) telle que la différence entre la somme des entiers de S_1 et celle de S_2 est égale à 1.

En effet, lorsque la somme totale est impaire, il est impossible de partager les entiers en deux sacs ayant exactement la même somme. La meilleure partition possible est donc celle où les deux sommes diffèrent de 1.

Exemple – Partition parfaite d'un sac impair

Le sac $S = \{1, 2, 3, 4, 5\}$ est impair car $1 + 2 + 3 + 4 + 5 = 15$.

- $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$ ne forment pas une partition parfaite, car $2 + 4 = 6$ et $1 + 3 + 5 = 9$.
- $S_1 = \{1, 2, 5\}$ et $S_2 = \{3, 4\}$ forment une partition parfaite, car $1 + 2 + 5 = 8$ et $3 + 4 = 7$, et la différence entre les deux sommes est 1.

Définition 1.5 – Problème de partitionnement

En informatique, le problème de partitionnement consiste à déterminer si une partition parfaite d'un sac d'entiers existe.

Ce problème est connu pour être difficile en informatique théorique, classé *NP-complet* :

- on peut vérifier rapidement si une partition candidate est correcte ;
- mais trouver une partition parfaite pour n'importe quel sac peut être très difficile.

Cependant, plusieurs algorithmes permettent de le résoudre de manière efficace pour des instances de taille raisonnable, soit exactement, soit approximativement. Pour cette raison, on l'appelle parfois :

le plus facile des problèmes difficiles.

Remarque 1.1 Symétries du problème

Ce problème admet une symétrie évidente puisque l'on peut inverser la partition, c'est-à-dire échanger les ensembles S_1 et S_2 .

De manière générale, on peut échanger des objets entre S_1 et S_2 tant que les sommes des deux sacs restent identiques.

Exemple – Symétries du problème

soit le sac $S = \{1, 2, 4, 6, 8, 9\}$ et la partition parfaite de somme égale à 15.

$$S_1 = \{6, 9\}, \quad S_2 = \{1, 2, 4, 8\}.$$

Inversion de la partition On peut inverser la partition et les sommes restent identiques.

$$S_1 = \{1, 2, 4, 8\}, \quad S_2 = \{6, 9\}$$

Échange partiel de nombres On peut aussi échanger des nombres sans changer les sommes, par exemple en remplaçant $6 \in S_1$ par $2, 4 \in S_2$.

$$S_1 = \{2, 4, 9\}, \quad S_2 = \{1, 6, 8\},$$

2 Partition parfaite des entiers de 1 à n

Exercice 2.1

Trouver une partition parfaite des entiers de 1 à n pour $n = 6, 7, 8, 9$.

Exercice 2.2

Trouvez d'autres partitions parfaites en effectuant des échanges partiels d'entiers entre les deux sacs.

Définition 2.1 – Algorithme

Un *algorithme* est une suite d'étapes simples qui permet de résoudre un problème. Le nombre d'étapes peut varier en fonction de la taille des données.

Remarque 2.1

- Plusieurs algorithmes peuvent répondre à un même problème.
- Un algorithme peut répondre à plusieurs problèmes.

Exercice 2.3

Donner un algorithme pour trouver une partition parfaite des entiers de 1 à n .

3 Algorithme glouton

Jusqu'à présent, nous avons manipulé des sacs d'entiers sans leur donner de sens concret. Pour appliquer les algorithmes, nous allons maintenant considérer chaque entier comme la *taille* d'un objet réel.

Ainsi, chaque entier du sac correspond à un objet que l'on peut placer dans un sac de capacité donnée. Dans les sections suivantes, nous utiliserons donc le terme *objet* tout en gardant à l'esprit qu'il sont seulement caractérisés par un entier, leur taille.

Définition 3.1 – Algorithme glouton

Un *algorithme glouton* est un algorithme qui, étape par étape, fait toujours le choix considéré comme le meilleur à ce moment précis (choix *localement optimal*). Dans certains problèmes, cette approche conduit à un optimum global. Dans le cas général, c'est une *heuristique* qui ne garantit pas de trouver l'optimum global.

Algorithme 3.1 – Algorithme glouton

- Déterminer la capacité du sac.
- Trier les objets par ordre décroissant de taille.
- Tant qu'il reste des objets à placer :
 - placer le plus grand objet dans le sac si sa capacité le permet ;
 - Sinon, retirer l'objet ;
 - Si le sac est rempli, arrêter l'algorithme.

Exercice 3.1

Appliquer l'algorithme glouton sur une instance de type 1.

#	Type	Taille	Entiers	Capacité
1	1	6	11, 8, 7, 5, 2, 1	17
2	1	6	16, 11, 10, 8, 4, 1	25
3	1	9	15, 13, 8, 7, 6, 5, 4, 2, 1	30
4	1	9	16, 12, 10, 9, 6, 5, 3, 2, 1	32
5	1	12	16, 15, 13, 12, 9, 8, 6, 5, 4, 3, 2, 1	47

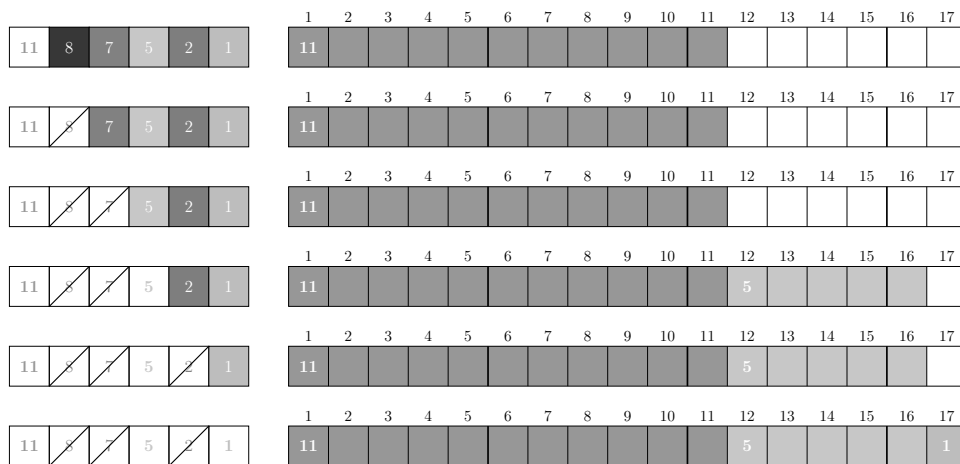


FIGURE 1 – Application de l'algorithme glouton sur l'instance #1 de type 1.

Exercice 3.2

Appliquer l'algorithme glouton sur une instance de type 2.

#	Type	Taille	Entiers	Capacité
6	2	6	14, 13, 11, 7, 5, 3	26
7	2	6	13, 12, 11, 10, 7, 4	28
8	2	9	16, 15, 14, 13, 12, 9, 8, 6, 1	47
9	2	9	15, 14, 13, 12, 11, 10, 7, 4, 3	44
10	2	12	16, 15, 13, 12, 11, 10, 8, 7, 6, 4, 3, 2	53

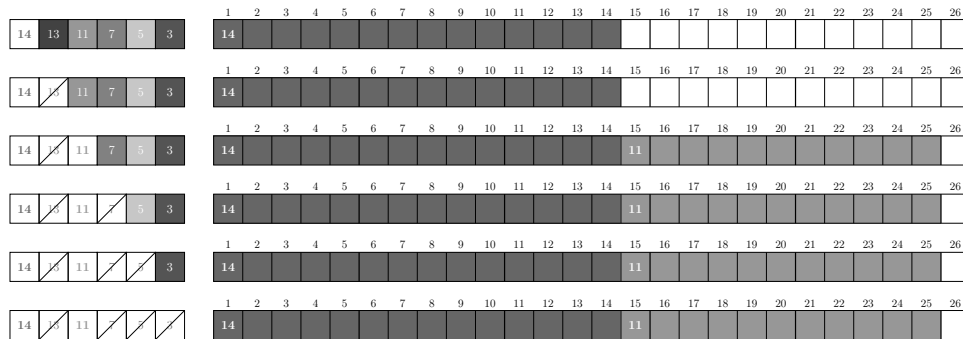


FIGURE 2 – Application de l'algorithme glouton sur l'instance #6 de type 2.

Remarque 3.1À propos de l'algorithme glouton

- On peut obtenir d'autres solutions en modifiant l'ordre dans lequel les objets sont considérés.
- Certains ordres différents donnent la même solution. Par exemple, échanger les objets 8 et 7 dans la solution #1 ne change pas la somme.
- Répéter le glouton avec plusieurs ordres différents permet de générer plusieurs solutions distinctes.

4 Algorithme glouton répété

Algorithme 4.1 – Algorithme glouton répété

Répéter tant que le sac n'est pas complètement rempli :

- Appliquer l'algorithme glouton.
- Si le sac est rempli, arrêter l'algorithme.
- Sinon, retirer le plus grand objet du sac restant et recommencer.

Exercice 4.1

Appliquer l'algorithme glouton répété sur une instance de type 2.

#	Type	Taille	Entiers	Capacité
6	2	6	14, 13, 11, 7, 5, 3	26
7	2	6	13, 12, 11, 10, 7, 4	28
8	2	9	16, 15, 14, 13, 12, 9, 8, 6, 1	47
9	2	9	15, 14, 13, 12, 11, 10, 7, 4, 3	44
10	2	12	16, 15, 13, 12, 11, 10, 8, 7, 6, 4, 3, 2	53

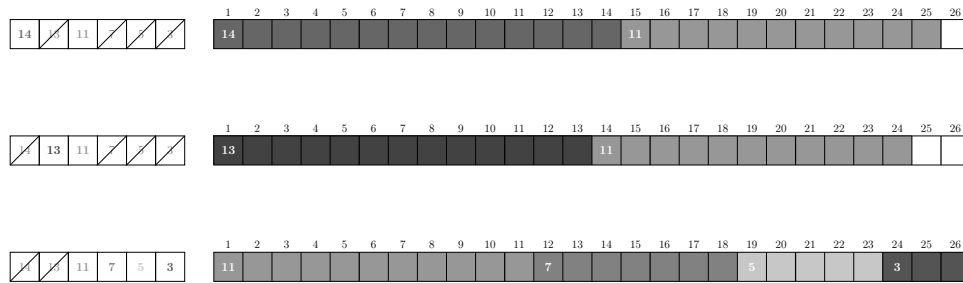


FIGURE 3 – Application de l'algorithme glouton répété sur l'instance #6 de type 2.

Exercice 4.2

Appliquer l'algorithme glouton répété sur une instance de type 3.

#	Type	Taille	Entiers	Capacité
11	3	6	13, 11, 9, 8, 6, 4	25
12	3	6	16, 13, 12, 11, 7, 3	31
13	3	9	16, 15, 14, 10, 9, 8, 6, 5, 3	43
14	3	9	16, 15, 13, 11, 9, 7, 5, 4, 3	41
15	3	12	16, 15, 14, 13, 12, 11, 10, 8, 7, 6, 4, 3	59

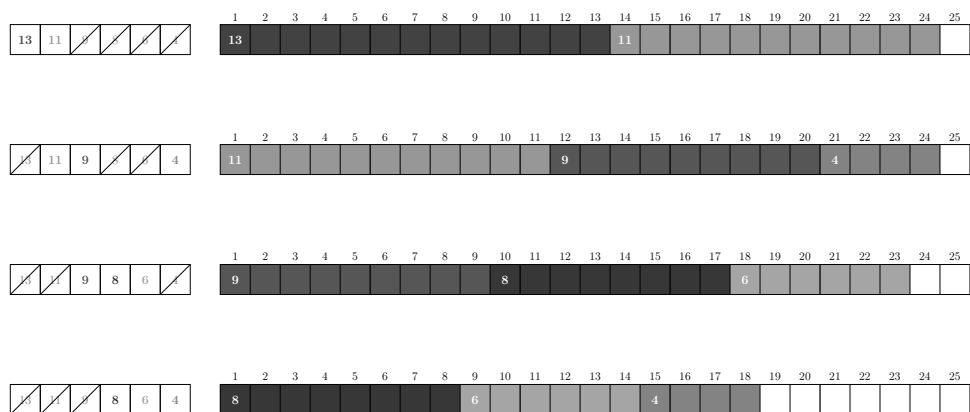


FIGURE 4 – Application de l'algorithme glouton répété sur l'instance #11 de type 3.

5 Programmation dynamique

Algorithme 5.1 – Algorithme de programmation dynamique

- Déterminer la capacité du sac.
- Trier les objets par ordre décroissant.
- Répéter tant qu'il reste des objets à placer :
 - Pour chaque case marquée, en partant de la dernière :
 - Calculer la case atteinte en plaçant immédiatement l'objet après la case marquée.
 - Si cette case n'est pas encore marquée, placer un marqueur correspondant à l'objet.
 - Si la dernière case (capacité) est marquée, le sac est rempli et l'algorithme s'arrête.

Algorithme 5.2 – Reconstruction du sac en programmation dynamique

- Tant qu'il reste des marqueurs dans le sac :
- Sélectionner le marqueur le plus à droite.
 - Placer l'objet correspondant à ce marqueur dans le sac en retirant les marqueurs sous l'objet.

Exercice 5.1

Appliquer la programmation dynamique sur une instance de type 3.

#	Type	Taille	Entiers	Capacité
11	3	6	13, 11, 9, 8, 6, 4	25
12	3	6	16, 13, 12, 11, 7, 3	31
13	3	9	16, 15, 14, 10, 9, 8, 6, 5, 3	43
14	3	9	16, 15, 13, 11, 9, 7, 5, 4, 3	41
15	3	12	16, 15, 14, 13, 12, 11, 10, 8, 7, 6, 4, 3	59

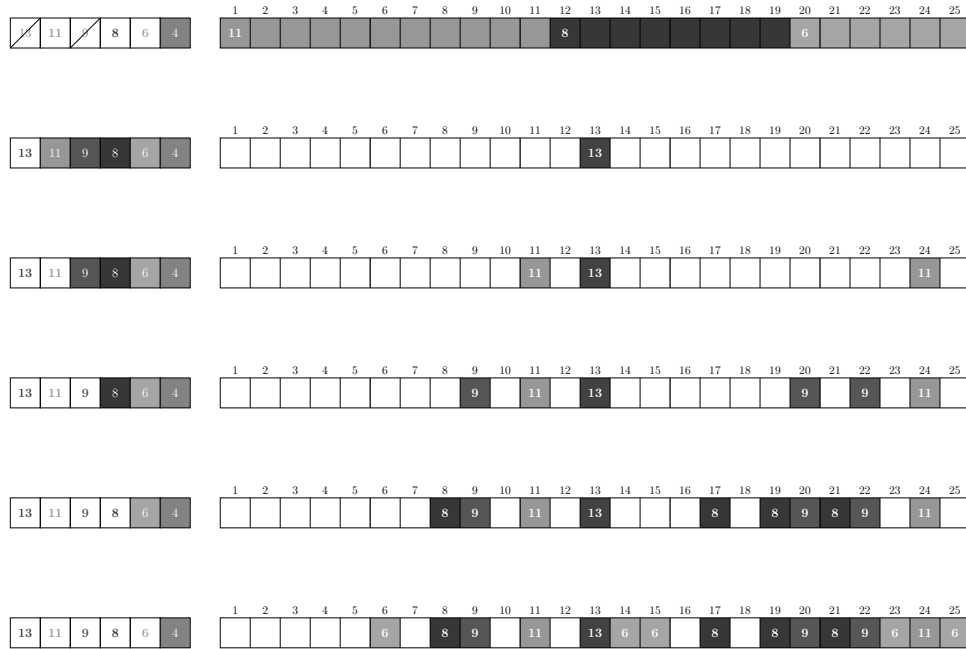


FIGURE 5 – Application de la programmation dynamique sur l'instance #11 de type 3.

Exercice 5.2

Appliquer la programmation dynamique sur une instance de type 4.

#	Type	Taille	Entiers	Capacité
16	4	6	16, 15, 11, 4, 2, 1	24
17	4	6	15, 14, 9, 8, 6, 1	26
18	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
19	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
20	4	8	18, 17, 16, 15, 14, 5, 2, 1	44

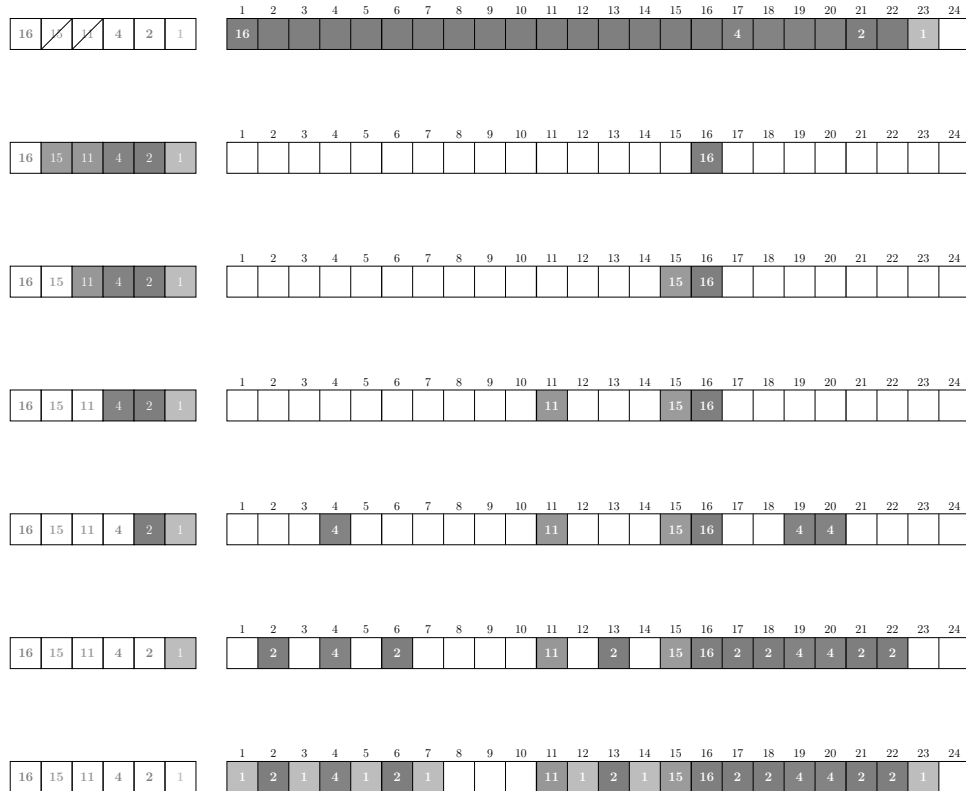


FIGURE 6 – Application de la programmation dynamique sur l'instance #16 de type 4.

6 Tester et Générer

Algorithme 6.1 – Algorithme Tester-et-Générer

- Déterminer la capacité.
- Trier les objets par ordre décroissant.
- Descente : Répéter tant que le dernier objet n'est pas dans le sac :
 - Répéter pour chaque objet :
 - ranger l'objet dans le sac si la capacité le permet.
 - Si le sac est rempli, arrêter l'algorithme.
 - Sinon, effectuer un retour arrière simple : retirer le plus petit objet du sac.
- Retour arrière sauté quand le plus petit objet du deck est dans le sac.
 - Retirer les objets du sac trouver jusqu'à ce que vous trouviez le plus petit objet que vous n'avez pas pu faire rentrer.
 - Sinon éliminer l'objet.

7 Problèmes connexes

#	Type	Taille	Entiers	Capacité
1	1	6	11, 8, 7, 5, 2, 1	17
2	1	6	16, 11, 10, 8, 4, 1	25
3	1	9	15, 13, 8, 7, 6, 5, 4, 2, 1	30
4	1	9	16, 12, 10, 9, 6, 5, 3, 2, 1	32
5	1	12	16, 15, 13, 12, 9, 8, 6, 5, 4, 3, 2, 1	47
6	2	6	14, 13, 11, 7, 5, 3	26
7	2	6	13, 12, 11, 10, 7, 4	28
8	2	9	16, 15, 14, 13, 12, 9, 8, 6, 1	47
9	2	9	15, 14, 13, 12, 11, 10, 7, 4, 3	44
10	2	12	16, 15, 13, 12, 11, 10, 8, 7, 6, 4, 3, 2	53
11	3	6	13, 11, 9, 8, 6, 4	25
12	3	6	16, 13, 12, 11, 7, 3	31
13	3	9	16, 15, 14, 10, 9, 8, 6, 5, 3	43
14	3	9	16, 15, 13, 11, 9, 7, 5, 4, 3	41
15	3	12	16, 15, 14, 13, 12, 11, 10, 8, 7, 6, 4, 3	59
16	4	6	16, 15, 11, 4, 2, 1	24
17	4	6	15, 14, 9, 8, 6, 1	26
18	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
19	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
20	4	8	18, 17, 16, 15, 14, 5, 2, 1	44

Allons plus loin

test

Références